Dodatak predavanjima: Imenici, biblioteke i zaglavlja (Razjašnjenja nekih nedoumica)

Motiv za pisanje ovog dodatka je veliki broj upita koji su već nakon prvog predavanja postavljeni predmetnom nastavniku vezanu za nedoumice vezane za imenike, biblioteke i zaglavlja. Imenici se obično koriste samo pri razvoju većih programa, tako da se početnici ne bi trebali zamarati s njima. Početnici uglavnom samo trebaju znati da postoji nekakav globalni imenik i nekakav imenik zvani "`std`", pri čemu su sve bibliotečke funkcije iz standardnih biblioteka u imeniku "`std`", dok su stvari koje programeri pišu u programu podrazumijevano u globalnom imeniku, ako se ne naglasi drugačije. Međutim, zbrka nastaje zbog činjenice da se već u najjednostavnijim programima koristi biblotečki imenik "`std`", što početnicima izaziva konfuziju po pitanju šta je tačno imenik, šta biblioteka, a šta zaglavlje biblioteke. Cilj ovog dodatka je da razjasni tu zbrku. Treba imati u vidu da, u osnovi, imenici i biblioteke kao pojmovi nemaju manje-više ništa zajedničko (druga je stvar što standardna biblioteka koristi imenik "`std`"). Stoga ćemo početi s imenicima.

Imenici nisu ništa drugo nego praktičan način hijearhijskog organiziranja imenovanih objekata poput funkcija, promjenljivih i raznih drugih objekata koji su dostupni u čitavom programu, s ciljem lakšeg snalaženja u velikim programima. Imenik se prosto započinje ključnom riječju "**`namespace`**" nakon kojeg ide ime imenika, te blok koji definira tijelo imenika. Sve što se definira u tijelu imenika, pripada tom imeniku. Ono što se ne definira ni u jednom imeniku, pripada globalnom imeniku. Ovo ilustrira sljedeći program, koji definira dva imenika "`A`" i "`B`", te ilustrira kako se imena iz tih imenika ispravno koriste, te šta nije ispravno:

```
#include <iostream>

namespace A {
  int x = 3;
  int y = 5;
  void f() { std::cout << "f iz A\n"; }
  void g() { std::cout << "g iz A\n"; }
};

namespace B {
  int y = 1;
  int z = 4;
  void f() { std::cout << "f iz B\n"; }
};

int z = 2;
void g() { std::cout << "g globalni\n"; }

int main() {
  std::cout << x << std::endl;      // Greška: "x" nije direktno vidljiv
  std::cout << A::x << std::endl;   // OK, ispisuje "3"
  std::cout << z << std::endl;      // OK, ispisuje "2"
  std::cout << B::z << std::endl;   // OK, ispisuje "4"
  using A::x;                       // "x" iz imenika "A" je sad direktno vidljiv
  std::cout << x << std::endl;      // OK, ispisuje "3"
  f();                              // Greška: "f" nije direktno vidljv
  A::f();                           // OK, ispisuje "f iz A"
  g();                              // OK, ispiisuje "g globalni"
  A::g();                           // OK, ispisuje "g iz A"
  using namespace A;                // Sve iz imenika "A" je sad direktno vidljivo
  f();                              // OK, ispisuje "f iz A"
  g();                              // Greška, poziv "g" je dvosmislen
  ::g();                            // OK, ispisuje "g globalni"
  A::g();                           // OK, ispisuje "g iz A"
  using namespace B;                // Sad je i sve iz imenika "B" direktno vidljivo
  std::cout << y << std::endl;      // Greška, pristup "y" je dvosmislen
  std::cout << A::y << std::endl;   // OK, ispisuje "5"
  std::cout << B::y << std::endl;   // OK, ispisuje "1"
  std::cout << z << std::endl;      // Greška, pristup "z" je dvosmislen
  std::cout << ::z << std::endl;    // OK, ispisuje "2"
  std::cout << B::z << std::endl;   // OK, ispisuje "4"
  return 0;
}
```

Svim identifikatorima iz nekog imenika možemo pristupiti navođenjem imena imenika, zatim operatora pristupa "`::`", a zatim navođenjem samog identifikatora. Za pristup globalnom imeniku (koji se podrazumijeva, osim u slučaju dvosmislica), koristi se samo opertator "`::`", bez ičega ispred. Direktiva "**`using`**" omogućava direktan pristup određenim identifikatorima iz određenih imenika, ili svim identifikatorima iz određenih imenika (u kombinaciji s "**`namespace`**"), osim u slučaju dvosmislica. Napomenimo da ukoliko se "**`using`**" direktiva ne navede globalno, nego unutar nekog bloka (kao u prikazanom programu), ona vrijedi samo "lokalno", unutar tog bloka. Inače, imenici mogu također biti i ugniježdeni, tako da unutar imenika "`A`" možemo imati novi imenik nazvan recimo "`C`". Potpuni pristup identifikatoru "`x`" iz tog imenika dobio bi se konstrukcijom "`A::C::x`", a tada su moguće i direktive poput "**`using`** `A::C::x`" ili "**`using namespace`** `A::C`".

Sad, nakon ove priče o imenicima, možemo reći i šta su tačno biblioteke. Biblioteke nisu ništa drugo nego već pripremljene kolekcije funkcija, definicija novih tipova, te raznih objekata spremnih za korištenje u drugim programima. Biblioteke najčešće dolaze u obliku gdje su funkcije i objekti definirani u biblioteci već kompajlirani, što ima dvije prednosti. Prvo, prilikom kompajliranja programa koji koristi te bibiloteke, ne gubi se vrijeme na ponovno kompajliranje bibliotečki definiranih stvari, jer su one već iskompajlirane. Drugo, iz kompajliranog koda nije moguće saznati kako je neka funkcija napisana (osim možda *reverznim inžinjeringom*, odnosno analizom napisanog mašinskog kôda), što omogućava autorima biblioteke da zaštite implementacije svojih funkcija od tuđih pogleda (recimo, ne možete na razuman način otkriti kako je napisana recimo funkcija "`sqrt`" iz biblioteke s zaglavljem "`cmath`"). Međutim, šta su uopće zaglavlja? Problem je što ukoliko je biblioteka u kompajliranoj formi, u takvoj formi su izgubljene brojne informacije bitne kompajleru, poput kako se funkcija zove, kakve parametre prima, koji je tip povratne vrijednosti, itd. (komapjirani mašinski kôd ne sadrži takve informacije, nego samo instrukcije koje daju postupak izvršavanja funkcije). Zbog toga uz svaku biblioteku dolaze tzv. *zaglavlja*. Zaglavlja su datoteke koje tipično sadržavaju samo deklaracije (odnosno prototipove) funkcija koje su definirane u biblioteci (bez njihove implementacije), zatim deklaracije tipova koje biblioteka definira, te eventualno još neke neophodne informacije. Uključenjem zaglavlja željene biblioteke u program, kompajler dobija neophodne informacije koje nije moguće dobiti iz kompajliranog kôda biblioteke. Sâmo uključivanje zaglavlja vrši se pomoću preprocesorske direktive "`#include`", koja prosto vrši umetanje sadržaja odgovarajuće datoteke na mjesto gdje je navedena. Recimo, kada stavimo "`#include <cmath>`" negdje u program, dobijamo isti efekat kao da smo prosto pretipkali čitav sadržaj zaglavlja "`cmath`" na to mjesto u program. Imena zaglavlja mogu biti ili u šiljastim zagradama "`<>`", ili između navodnika (recimo, legalno je i "`#include "cmath"`"). Razlika je u tome što ako se upotrijebe šiljaste zagrade, zaglavlje se traži isključivo u folderima predviđenim za sistemska zaglavlja (koji su to folderi, određeno je postavkama kompajlera), a ako se iskoriste navodnici, zaglavlje se prvo traži u tekućem folderu (u kojem se nalazi i sam program), a tek ukoliko se ne nađe tamo, onda se traži u sistemskim folderima. Stoga se šiljaste zagrade tipično koriste za sistemska zaglavlja, a navodnici za zaglavlja koja pravi sam programer za svoje potrebe.

Biblioteke koje dolaze u kompajliranoj formi malo su komplikovanije za upotrebu, jer je kompajleru potrebno nekako saopštiti gdje se nalazi mašinski kôd stvari definiranih u toj biblioteci (sama direktiva "`#include`" nije dovoljna, jer ona "uvozi" samo zaglavlje, koje tipično ne sadrži definicije funkcija i objekata, nego samo njihove deklaracije). To se radi negdje u postavkama kompajlera (ovo ne treba raditi za standardne biblioteke, jer kompajler već dolazi konfigurisan tako da "zna" gdje se definicije funkcija i objekata iz standardne biblioteke nalaze). Početnicima to stvara dodatne komplikacije kadgod hoće da koriste neku od nestandardnih biblioteka. Stoga su u posljednje vrijeme popularne biblioteke koje se sastoje *samo od zaglavlja*, tzv. *header-only* biblioteke. Kod tih biblioteka, zaglavlje sadrži ne samo deklaracije, nego i kompletne definicije svih funkcija i objekata koje se u njima nalaze. Dakle, kod njih, sve što nam treba u potpunosti se nalazi u zaglavlju, tako da je dovoljna samo direktiva "`#include`"). Jasno je koje su mane ovih biblioteka. Na prvom mjestu, sporije kompajliranje, jer se svaki put kada koristimo tu biblioteku iznova komapjliraju i stvari koje su u njoj definirani. Drugo, svak može otvoriti odgovarajuće zaglavlje i vidjeti kompletnu definiciju svih stvari koje ona definira (odnosno, nije moguće "sakriti" njihov sadržaj od neželjenih očiju). Pored toga, takve biblioteke je i malo teže pisati, zbog tzv. ODR (od engl. *one definition rule*) pravila, koje propisuje da se nijedna funkcija ni objekat ne može *definirati* na više od jednog mjesta u programu, iako su višestruke *deklaracije* iste stvari dopuštene (npr. ne možemo imati dvije definicije funkcije istog imena i istih parametara u programu, čak i ako im je tijelo potpuno isto). Ovo može napraviti problem ukoliko recimo dvije header-only biblioteke "`A`" i "`B`" koriste treću header-only biblioteku "`C`" (jedna biblioteka naravno i sama može koristiti druge biblioteke). Tada bi istovremeno uključenje biblioteka "`A`" i "`B`" u program dovelo do dvostrukog

definiranja svih stvari definiranih u biblioteci "`C`", što dovodi do pojave greške od strane kompajlera. Postoje relativno jednostavni trikovi kako se ipak mogu napraviti header-only biblioteke, a da ne dođe do kršenja ODR pravila (ti trikovi zasnivaju se na adekvatnom korištenju ključne riječi "**`inline`**", uz pomoć koje su, pod određenim uvjetima, višestruke definicije ipak dopuštene). U detalje ipak nećemo ulaziti, jer traže malo više znanja.

Kao što vidimo, imenici i biblioteke su dvije *posve različite stvari*. Druga je stvar što većina biblioteka, upravo radi izbjegavanja konflikta, kreira neki imenik i sve stvari čuva u njemu. Pri čemu je za sve standardne biblioteke to imenik koji se zove "`std`". Dakle, većina biblioteka zaista kreira i neki imenik karakterističan za nju, ali to ovisi od autora biblioteke. Biblioteku može i sam napraviti bilo koji programer koji ima dovoljno znanja za to, i njega tebe je kako će je napraviti, odnosno da li će u njoj biti korišteni imenici ili ne. Smatra se kulturnim koristiti imenike, pogotovo ako se planira da će tu bibiloteku koristiti i drugi programeri, pogotovo ako će je koristiti mnogo programera. Ukoliko autor biblioteke nije koristio imenike, sve će biti u globalnom imeniku. A kao što je već rečeno, za sve standardne biblioteke, svi identifikatori su uvijek u imeniku "`std`". Izuzetak su biblioteke naslijeđene iz jezika C za koje se dopušta (ali ne garantira) da će njihovi identifikatori pored imenika "`std`" biti i u globalnom imeniku. Tačnije, autori biblioteke stave recimo zaglavlje funkcije "`sqrt`" u imenik "`std`", ali u biblioteku dodaju i direktivu "**`using`** `std::sqrt`" i na taj način učine automatski funkciju "`sqrt`" dostupnu kao da je u globalnom imeniku. Rečeno je da je ovo ponašanje predviđeno kao "privremeno rješenje" i standardom je dopušteno, ali nije garantirano, te se na to ne treba osloniti.

Nažalost, i dalje se viđaju mnoge nestandardne biblioteke koje drže sve u globalnom imeniku, pogotovo ukoliko su ih pravili amateri. Ukoliko koristimo dvije biblioteke koje ne koriste imenike, odnosno čiji svi identifikatori pripadaju globalnom imeniku, imaćemo nerješiv problem kadgod probamo pristupiti nekom identifikatoru koji postoji u obje biblioteke. Srećom, kvalitetni autori biblioteka vode računa o ovome. Recimo, Boost "standardna kolekcija nestandardnih biblioteka" u svim svojim bibliotekama insistira da njihovi autori stavljaju sve u imenik koji se zove "`boost`". Kako imenici mogu biti i ugniježdeni, mogu se viđati stvari poput "`boost::parse::get_token()`" ili direktive poput "**`using`** `boost::parse::get_token`" ili "**`using namespace`** `boost::parse`".

Predmetni nastavnik se nada da će nakon ovoga izlaganja, stvari vezane za imenike, biblioteke i zaglavlja biti malo jasnije.